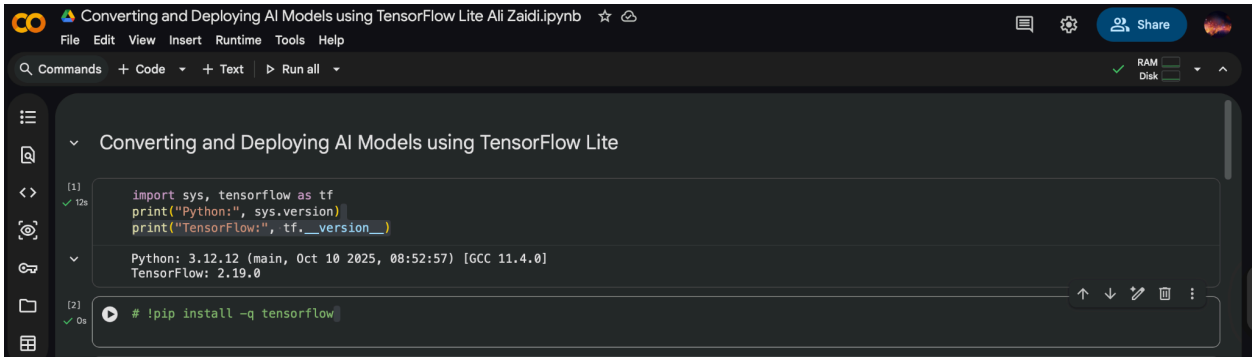
1. Lab Objective

The purpose of this lab is to train a simple neural network on the MNIST dataset using TensorFlow/Keras, convert the trained model into TensorFlow Lite (.tflite) format, and run inference using the TensorFlow Lite Interpreter. This demonstrates a typical workflow used to deploy models onto resource-constrained devices such as mobile phones and embedded/IoT hardware.



---

2. Environment Setup and Verification

Step 1: Verify Python and TensorFlow Installation

I verified the Python version and TensorFlow installation to ensure the environment supports training and TensorFlow Lite conversion.

What I checked:

- Python version
- TensorFlow version (ensuring tensorflow is installed)

```
import sys, tensorflow as tf
print("Python:", sys.version)
print("TensorFlow:", tf.__version__)
```

```
Python: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
TensorFlow: 2.19.0
```

---

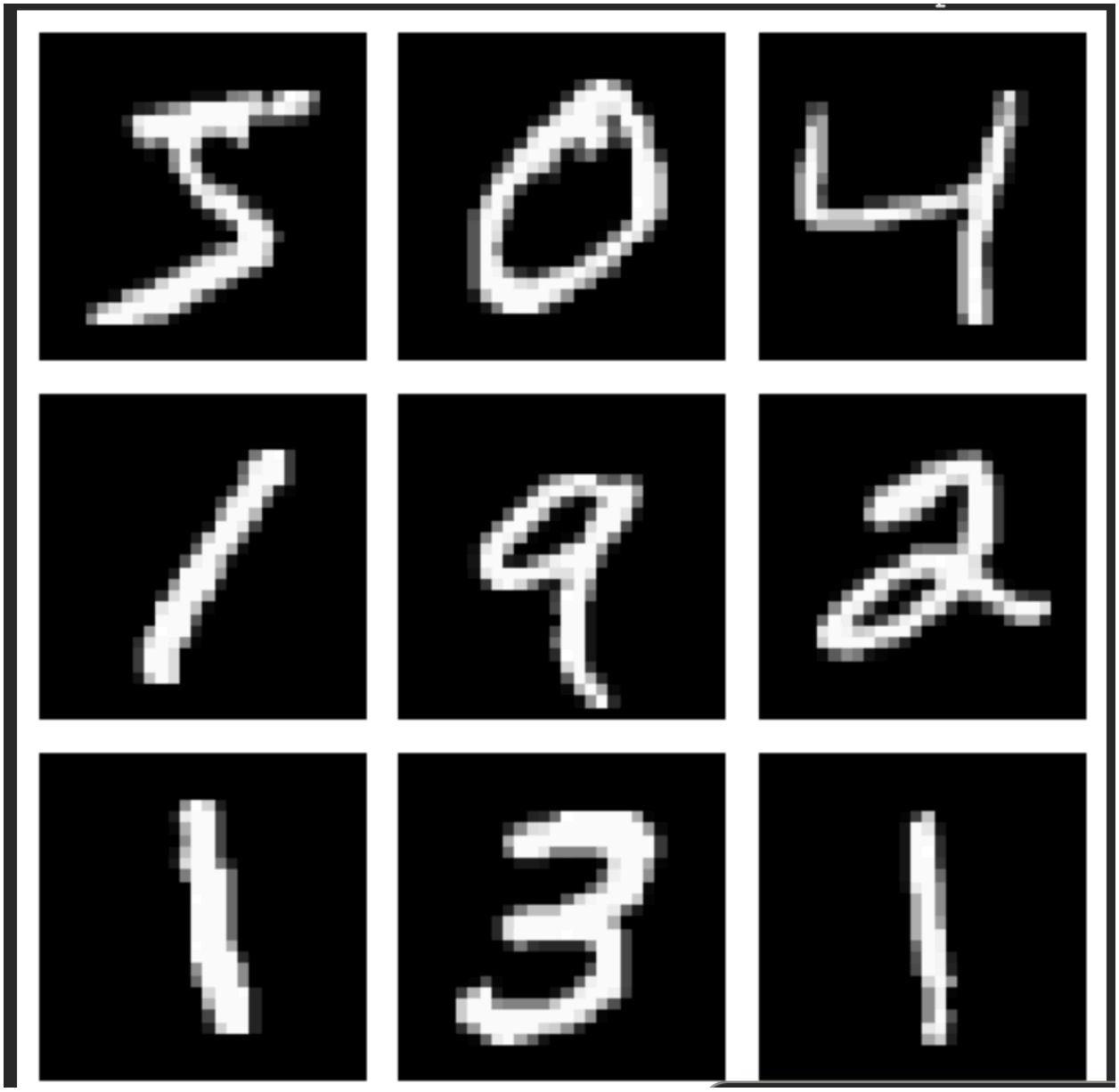
3. Dataset Loading and Preprocessing

Step 3: Load the MNIST Dataset

MNIST is a dataset of 28×28 grayscale images of handwritten digits (0–9). I loaded the training and test splits using tensorflow.keras.datasets.mnist.

Normalization

The pixel values were normalized from [0, 255] to [0, 1] by dividing by 255.0. This helps neural networks train faster and more reliably because the input scale is consistent.



---

4. Model Design and Training

Step 4: Define and Train a Neural Network

I used a simple feedforward neural network with:

- Flatten Layer: Converts 28×28 input into a single vector (784 features)
- Dense Hidden Layer (128 units, ReLU): Learns patterns/features from the pixels
- Dense Output Layer (10 units, Softmax): Produces probabilities for 10 digit classes

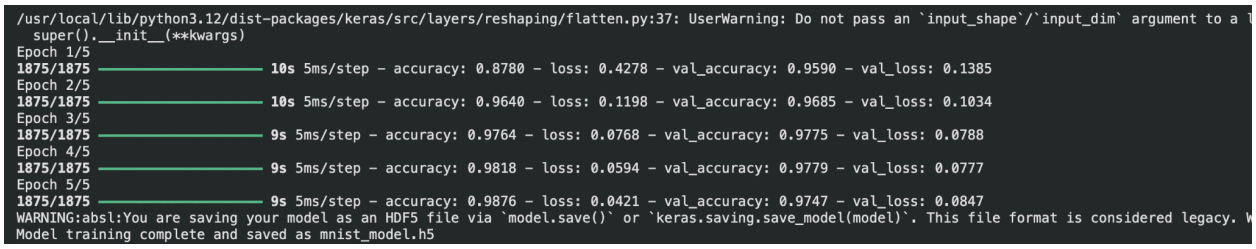
Compilation

The model was compiled using:

- Optimizer: Adam (adaptive learning rate optimizer)
- Loss: Sparse Categorical Crossentropy (appropriate for integer labels)
- Metric: Accuracy

Training

The model was trained for 5 epochs, and validation was performed using the test set.



---

5. Converting the Model to TensorFlow Lite

Step 5: Convert and Save as .tflite

After training, I loaded the saved .h5 Keras model and converted it using tf.lite.TFLiteConverter.from_keras_model(model). The resulting model was saved as mnist_model.tflite.

Why convert to TFLite?
TensorFlow Lite models are optimized for:

- smaller model size
- faster inference
- compatibility with mobile/embedded devices

```
Model successfully converted and saved as mnist_model.tflite
```

---

6. Loading the TFLite Model and Running Inference

Step 6: Load the Model with TensorFlow Lite Interpreter

I loaded the .tflite model using tf.lite.Interpreter, allocated tensors, and printed input/output details to verify the model is correctly loaded.

```
Input Details: [{'name': 'serving_default_input_layer:0', 'index': 0, 'shape': array([ 1, 28, 28], dtype=int32), 'shape_signature': array([-1, 28, 28],
Output Details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 9, 'shape': array([ 1, 10], dtype=int32), 'shape_signature': array([-1, 10], dtype=in
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:     Warning: tf.lite.Interpreter is deprecated and is s
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

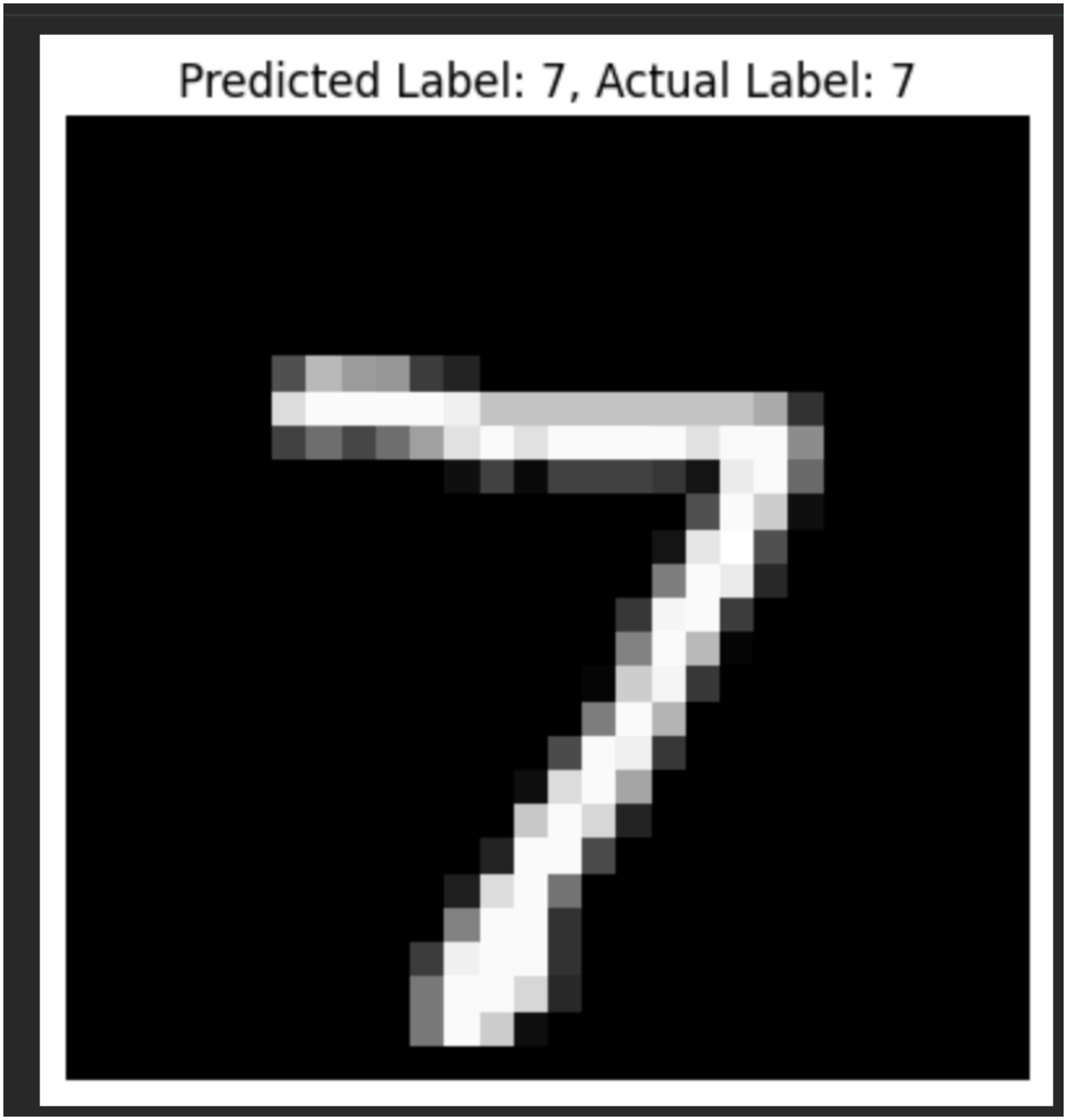
Step 7: Perform Inference

I selected a test image, reshaped it to include a batch dimension (1, 28, 28), and ran inference using:

- set_tensor()
- invoke()
- get_tensor()

Finally, I displayed the test image with:

- predicted label
- actual label



---

7. Conclusion

This lab successfully demonstrated an end-to-end workflow:

1. Train a Keras model on MNIST
2. Convert it into TensorFlow Lite format
3. Load it using the TFLite interpreter

4. Run inference on real test data

This process mirrors how models are deployed for real-world AI applications on edge devices.